5.Write a Java program to

a) demonstrate exception handling

Type	Input	Expected Output	Explanation
Success	10 / 2	Result = 5	Normal division
Success	10 / 0	ArithmeticException caught	Division by zero
Success	Array index within range	Value displayed	Normal array access
Failure/Edge	Invalid array index	ArrayIndexOutOfBoundsException caught	Boundary error
Failure/Edge	Input abc for integer	NumberFormatException caught	Invalid input

Algorithm:

1. **Start** the program.
2. Create a `try` block.
3. Perform division of two numbers and display the result.
4. Create an array and display an element.
5. Convert a string `"abc"` into an integer.
6. If an **ArithmeticException** occurs, display the corresponding message.
7. If an **ArrayIndexOutOfBoundsException** occurs, display the corresponding message.
8. If a **NumberFormatException** occurs, display the corresponding message.
9. **Stop** the program.

Program:

```
public class ExceptionHandlingDemo {
   public static void main(String[] args) {
      // Test Case 1: Normal Division
      try {
         int a = 10;
         int b = 2;
         int result = a / b;
         System.out.println("Result = " + result);
                  int arr[] = {10, 20, 30, 40};
         System.out.println("Value displayed: " + arr[3]);
                  String str = "abc";
         int num = Integer.parseInt(str);
         System.out.println(num);
                  } catch (ArithmeticException e) {
         System.out.println("ArithmeticException caught");
       }
         catch (ArrayIndexOutOfBoundsException e)
         {
           System.out.println("ArrayIndexOutOfBoundsException caught");
      }
         catch (NumberFormatException e) {
```

```
            System.out.println("NumberFormatException caught");
        }
}
}
```

b) create user-defined exception

Type	Input	Expected Output	Explanation
Success	Age=20	Valid age	Normal input
Success	Marks=75	Valid marks	Valid range
Success	Balance=5000	Valid balance	Normal condition
Failure/Edge	Age=15	Custom exception: Invalid age	User-defined exception
Failure/Edge	Marks=-10	Custom exception: Invalid marks	Range validation

Algorithm:

1. **Start** the program.
2. Create custom exceptions `InvalidAgeException` and `InvalidMarksException`.
3. Define `validateAge()` to check whether age is at least 18.
4. If age is less than 18, **throw `InvalidAgeException`**; otherwise display **Valid age**.
5. Define `validateMarks()` to check whether marks are between 0 and 100.
6. If marks are invalid, **throw `InvalidMarksException`**; otherwise display **Valid marks**.
7. Define `validateBalance()` to check whether the balance is non-negative.
8. In `main()`, call the validation methods using `try-catch` blocks.
9. Catch the custom exceptions and display their messages.
10. **Stop** the program.

Program:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
class InvalidMarksException extends Exception {
    public InvalidMarksException(String message) {
        super(message);
    }
}
public class UserDefinedExceptionDemo {

    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Custom exception: Invalid age");
        } else {
            System.out.println("Valid age");
        }
    }
   static void validateMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
```

```
            throw new InvalidMarksException("Custom exception: Invalid marks");
        } else {
            System.out.println("Valid marks");
        }
    }
    static void validateBalance(double balance) {
        if (balance >= 0) {
            System.out.println("Valid balance");
        }
    }
    public static void main(String[] args) {

        // Test Case 1: Age = 20
        try {
            validateAge(20);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
        // Test Case 2: Marks = 75
        try {
            validateMarks(75);
        } catch (InvalidMarksException e) {
            System.out.println(e.getMessage());
        }
        // Test Case 3: Balance = 5000
        validateBalance(5000);

        // Test Case 4: Age = 15
        try {
            validateAge(15);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
        // Test Case 5: Marks = -10
        try {
            validateMarks(-10);
        } catch (InvalidMarksException e) {
            System.out.println(e.getMessage());
        }
    }
}
```